中山大學
SUN YAT-SEN UNIVERSITY

继承和派生（中）

中山大学计算机学院

主讲人：？？？

？？？@mail.sysu.edu.cn

目录

CONTENTS

01 继承/派生的概念

02 语法与访问控制

03 构造与析构顺序

04 派生与构造函数

05 派生与成员函数

06 改变访问控制

07 类型兼容性

08 类的类型转换

09 多重继承

Overview: 类不可继承的成员

- 类有哪些成员不可继承：
  - 私有成员
  - 构造函数与析构函数
- ex1：
  - string的构造并不能使用
- ex2：
  - 如何在 derive 类中正确初始化 base 类
  - string 类重载的运算符被继承了？

运算符重载是否被继承是个复杂话题，注意回避哦！

```
#include <string>
#include <iostream>

class MyString : public std::string {
    public:
//      MyString(const char* s): std::string(s) {};
};

int main() {
    //ex1: 编译，并观看编译日志
    MyString str1("继承 string(const *char)");
    str1 = "继承 string::operator=(...)";
    std::cout << str1 << std::endl;
    MyString str2 = "hello ";
    std::cout << str2 + str1 << std::endl;
    //ex2: 取消第 6 行注释，编译并运行
    return 0;
}
```

向基类构造函数传递实参

- 若基类构造函数带参数，则定义派生类构造函数时，仅能通过初始化列表显式调用基类构造函数，并向基类构造函数传递实参。
- 带初始化列表的派生类构造函数的一般形式如下

```
派生类名(形参表) : 基类名(实参表)
{
    …;
};
```

向基类构造函数传递实参举例：Time类

```
//  SPECIFICATION FILE (time.h)

class Time
{
public :
       void Set(int hours, int minutes, int seconds);
       void Increment();
       void Write() const;
       Time(int initHrs, int initMins, int initSecs);  // constructor
       Time();           //  default constructor

private :
       int hrs;
       int mins;
       int secs;
} ;
```

将被修改

将被继承

向基类构造函数传递实参举例：Time类

```
// SPECIFICATION   FILE                          ( exttime.h)
#include “time.h”
enum  ZoneType {EST, CST, MST, PST, EDT, CDT, MDT, PDT } ;

class  ExtTime: public Time  // Time is the base class
{
public :
        ExtTime(int initHrs, int initMins, int initSecs,
                 ZoneType initZone ) ;      // constructor
        ExtTime() ;                                  // default
constructor
        void Set(int hours, int minutes, int seconds,
                          ZoneType timeZone ) ;
        void Write() const ;

private :
       ZoneType  zone ;      //  added data member
} ;
```

自学：枚举类型

派生类构造函数

扩展函数成员

增加数据成员

向基类构造函数传递实参举例：Time类

```
// IMPLEMENTATION FILE (exttime.cpp)

ExtTime::ExtTime(int initHrs, int initMins, int initSecs, ZoneType
initZone ): Time(initHrs, initMins, initSecs)
{
      zone = initZone;
}
```

初始化基类的成员

初始化派生类成员

1. 传递给基类构造函数。
2. 基类构造函数在派生类构造函数之前调用。

向基类构造函数传递实参举例：Time类

```
//base class default constructor is called prior to the derived //class
default constructor.

ExtTime::ExtTime()
{
    zone = EST;
}
```

编译器默认添加了 `:Time()` 初始化基类成员

如果枚举值标识符重名，请使用类型::枚举值。例如：ZoneType::EST

```
void ExtTime::Set( int hours,
                   int minutes,
                   int seconds,
                   ZoneType timeZone )
{
    Time::Set(hours, minutes, seconds);   //调用基类函数。Why?
    zone = timeZone;
}

void ExtTime::Write() const
{
    static string zoneString[8] =
    {  "EST", "CST", "MST", "PST", "EDT", "CDT", "MDT", "PDT" };

    Time::Write();
    cout << ' ' << zoneString[zone];
}
```

练习：去掉 Time::编译的结果？

同名 基类、派生类都有Set

数组总比 switch 语句好！

目录

CONTENTS

01 继承/派生的概念

02 语法与访问控制

03 构造与析构顺序

04 派生与构造函数

05 派生与成员函数

06 改变访问控制

07 类型兼容性

08 类的类型转换

09 多重继承

派生与成员函数

- 重载（overload）
  (1)具有相同的作用域(即同一个类定义中)；
  (2)函数名字相同
  (3)参数类型(包括const 指针或引用），顺序 或 数目不同
- 覆盖（override）- **修改**基类函数定义（后面课程讲）
- 隐藏（overwrite）- **屏蔽**基类的函数定义
  (1) 派生类的函数与基类的函数同名，但是参数列表有所差异。
  (2) 派生类的函数与基类的函数同名，参数列表也相同，但是基类函数没有virtual关键字。
- 继承
  (1) 没有被覆盖或隐藏的基类函数，包括在基类中重载的函数

重载和隐藏的区别

- 例如：
  - Time::Set(int,int,int)
  - ExtTime::Set(int,int,int,ZoneType)
- 如果 Set 在一个类中定义，则是重载
  - Set 的函数签名不一样
- 如果**同名**函数出现在 Base 和 Derived 中，且满足特征(1)或(2)
  - ExtTime案例 Set 满足特征（1）
  - ExtTime案例 Write 满足特征（2）

  所以，它们都属于隐藏

Base类成员函数Derived类中被隐藏与显式调用

- 阅读 97~109 行代码，关于 ExtTime::Set 和 ExtTime::Write 的定义
- **在 Derived 或IndirectDerived调用被隐藏的函数成员**
  - Base::HidenFun(...) 或 this-> Base::HidenFun(...)
- 阅读 134~137 行代码，关于 Client 代码
- **在 client中 Derived 类对象 d 调用被隐藏的函数成员**
  - d.Base::HidenFun(...) 或 pd-> Base::HidenFun(...)
- **练习：** 在 116 行，插入语句 time1.Set(1,1,1);
  - 解释报告的编译错误

隐藏的应用

- 利用隐藏，在派生函数中修改成员函数的功能，如 Write。
- 利用隐藏，赋予派生类成员函数新的功能。

```
//  IMPLEMENTATION FILE ( time.cpp )
void Time::Set( int hours, int minutes, int seconds )

{
    hrs = hours;
    mins = minutes;
    secs = seconds;
}
```

```
//  IMPLEMENTATION FILE ( Exttime.cpp )
void ExtTime::Set( int hours, int minutes, int seconds,
                                   ZoneType timeZone )
{
    Time::Set(hours, minutes, seconds);
    zone = timeZone;
}
```

继承中的对象访问举例

```
class BASE {
    public:
        BASE(int p1, int p2);
        int inc1();
        int inc2();
        void display();

     private:
        int mem1, mem2;
};
```

没有默认构造！！！

被继承

```
#include “base.h”

class DERIVED : public BASE{
    public:
        DERIVED(int x1, int x2, int x3, int x4, int x5);
        int inc1() ;
        int inc3( ) ;
        void display( ) ;

    private:
        int mem3;
        BASE mem4;
};
```

新添成员

隐藏（规则2），修改定义

类成员（注意初始化方法）

继承中的对象访问举例

```
#include “BASE.h”

BASE::BASE(int p1, int p2)
{ mem1 = p1; mem2 = p2;}

int BASE::inc1() { return ++mem1; }

int BASE::inc2() { return ++mem2; }

void BASE::display()
{
      cout << "mem1 = “ << mem1
           << ", mem2 = “ << mem2 << endl;
}
```

```
#include “Derived.h”
DERIVED::DERIVED(int x1, int x2, int x3, int x4, int x5):
BASE(x1,x2), mem4(x3,x4)
{ mem3 = x5 ; }

int DERIVED::inc1() { return  BASE::inc1();}

int DERIVED::inc3( ) { return ++mem3 ; }

void DERIVED::display( )
{           BASE::display();
            mem4.display();
            cout<<"mem3 = "<<mem3<<"\n";    }
```

基类，类成员初始化

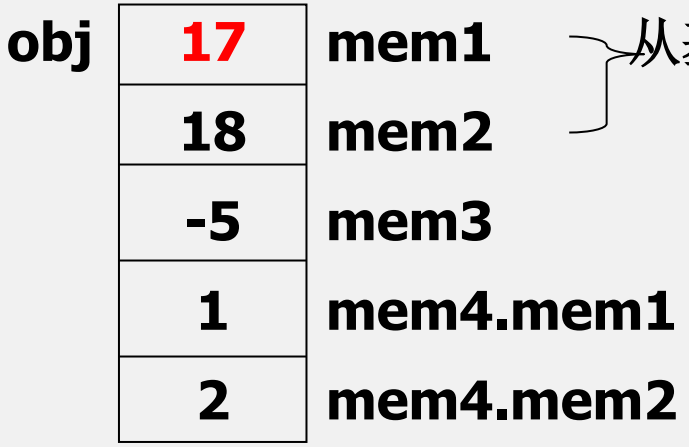
基类，类成员不能在这里初始化

被隐藏函数成员调用

继承中的对象访问举例（存储结构）

```
#include “Derived.h”
int main()
{
      DERIVED obj( 17, 18, 1, 2, -5);
      obj.inc1();
      obj.display();
      return 0;
}
```

mem1 = 18, mem2 = 18
mem1 = 1, mem2 = 2
mem3 = -5



执行DERIVED obj(17, 18, 1, 2, -5);之后

执行obj.inc1();之后

目录

CONTENTS

01 继承/派生的概念

02 语法与访问控制

03 构造与析构顺序

04 派生与构造函数

05 派生与成员函数

06 改变访问控制

07 类型兼容性

08 类的类型转换

09 多重继承

恢复访问控制方式

- 基类中的public或protected成员，因使用protected或private继承访问控制而导致在派生类中的访问方式发生改变，可以使用“访问声明”恢复为原来的访问控制方式
- 访问声明的形式

  using 基类名::成员名;（放于适当的成员访问控制后）
- 使用情景
  - 在派生类中希望大多数继承成员为protected或private，只有少数希望保持为基类原来的访问控制方式。

恢复访问控制方式

public变private.

```
class BASE {
public:
        void set_i(int x)
        {
          i = x;
        }
        int get_i()
        {
          return i;
        }
protected:
         int i;
};
```

```
class DERIVED: private BASE {
public:
        using BASE::set_i;    // 访问声明
        using BASE::i;
        void set_j(int x)
        {
          j = x;
        }
        int get_ij()
        {
          return i + j;
        }
protected:
        int j;
};
```

恢复访问控制方式

```
int main()
{
        DERIVED obj;        // 声明一个派生类的对象

        obj.set_i(5);       // set_i()已从私有的转为public
        obj.set_j(7);       // set_j()本来就是公有的
        cout << obj.get_ij() << "\n"; // get_ij()本来就是公有的

        return 0;
}
```

屏蔽基类成员

- 目的：
  - 使得客户代码通过派生类对象不能访问继承成员。
- 方法：
  - 使用继承访问控制protected和private（真正屏蔽）
  - 在派生类中成员访问控制 protected 或 private 之后，使用“using 基类名::成员名”（非真正屏蔽，仍可通过使用“基类名::成员名”访问）

用于继承对象的重命名

- 目的：
  - 解决名字冲突。
  - 在派生类中选择更合适的术语命名继承成员。
- 方法
  - 屏蔽旧函数（基类成员）
  - 在派生类中定义新的函数，该函数调用旧函数；或
  - 在派生类中定义新的函数，该函数的函数体与旧函数相同。

```
string str = “abc”;
cout << str.length();
cout << str.size();
```

使用基类构造函数

- 目的：
  - 使得派生类对象直接使用基类的构造函数。
- 方法：
  - 在派生类中使用 “using 基类名::基类名”
- 练习：
  - 打开 non_inheritable.cpp
  - 在 Mystring 声明中添加 using string::string;
  - 编译成功了，string的各种构造都延续（继承？）了

继承机制的应用举例——图形的处理

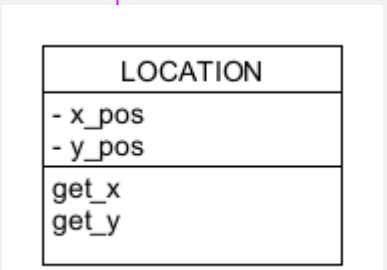
- 图形对象处理
  - 图形对象设计受系统和显示设备影响，不同系统差别较大
  - 课件内容仅供讲解继承机制，用于参考和学习，不一定是合理的设计
  - 一种可运行的实现版本，参见 graph_demo.cpp
    - 该版本需要 EGE 软件包支持
  - **课后任务：**
    - 设计并编写类 FiveStar
    - fivestar 对象可自由移动位置、改变大小、旋转角度、填充颜色等
    - 请参考 EGE 自带 demo/src/ege5star.cpp

继承机制的应用举例——图形的处理

- 图形元素的处理：将圆看作是一种带有半径的点，而将点看作是一种带有显示状态的位置。

BASGRAPH.H

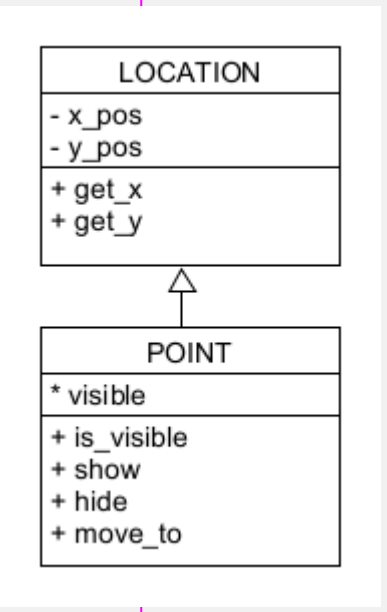
```
//说明：类LOCATION以x和y坐标描述了计算机屏幕上的一个位置。

#include <graphics.h>
class LOCATION {
public:
      LOCATION(int x, int y); // 构造函数，将当前位置设置为(x, y)

      int get_x();  // 返回当前位置的x坐标

      int get_y();  // 返回当前位置的y坐标

protected:
      // 位置的内部状态，在LOCATION的派生类中需要访问
       int x_pos, y_pos;
};
```



继承机制的应用举例——图形的处理

BASGRAPH.H

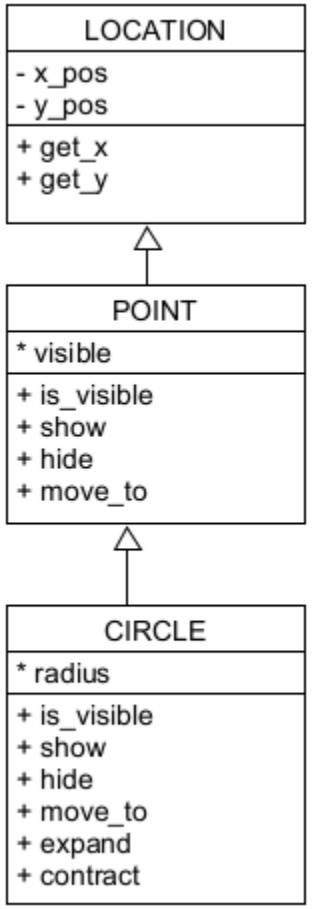
```
// 说明：类POINT描述了某一个位置是隐藏的还是显示的。
// 以public继承表示x_pos和y_pos在POINT中是protected
class POINT: public LOCATION {
public:
        POINT(int x, int y); // 构造函数，初始化位置为(x, y)

       // 判断当前点是否已显示，是则返回TRUE，否则返回FALSE
       BOOLEAN is_visible();
       void show();      // 在当前位置显示点
       void hide();        // 将点隐藏起来
       // 将当前点移动到新位置(x, y)并显示它
       void move_to(int x, int y);
protected:
      // 点的内部状态，在POINT的派生类中需要访问
       BOOLEAN visible;
};
```



继承机制的应用举例——图形的处理

BASGRAPH.H

```
// 说明：类CIRCLE描述了一个在屏幕上由POINT派生出来的圆。
// 由POINT类派生，从而也继承了LOCATION类
class CIRCLE: public POINT {
public:
        // 构造函数，初始化圆心为(x, y)，半径为r
        CIRCLE(int x, int y, int r);
        void show(); // 在屏幕上画出圆
        void hide(); // 将圆隐藏起来
        void move_to(int x, int y); // 将当前圆移动到新位置(x, y)
        // 放大圆，使得新的半径为(r + delta)
        void expand(int delta);
        // 缩小圆，使得新的半径为(r - delta)
        void contract(int delta);
protected:
        int radius;    // 圆的半径，在CIRCLE的派生类中可以访问
};
```



继承机制的应用举例——图形的处理

LOCATION.CPP

```
#include "basgraph.h"

LOCATION::LOCATION(int x, int y)
{
        x_pos = x;
        y_pos = y;
}

int LOCATION::get_x()
{
        return x_pos;
}

int LOCATION::get_y()
{
        return y_pos;
}
```

继承机制的应用举例——图形的处理

LOCATION.CPP

```
#include "basgraph.h"

POINT::POINT(int x, int y): LOCATION(x, y)
{
        visible = FALSE;  // 缺省情况下不显示
}

BOOLEAN POINT::is_visible()
{
        return visible;
}
```

继承机制的应用举例——图形的处理

LOCATION.CPP

```
void POINT::show()
{
       if (! is_visible()) {
           visible = TRUE;
        putpixel(x_pos, y_pos, getcolor()); // 使用
缺省颜色画点
       }
}

void POINT::hide()
{
      if (is_visible()) {
       visible = FALSE;
       // 使用背景颜色画点，即擦除该点
           putpixel(x_pos, y_pos, getbkcolor());
      }
}
```

继承机制的应用举例——图形的处理

LOCATION.CPP

```
void POINT::move_to(int x, int y)
{
        hide();                 // 首先使当前点不可见

        x_pos = x;      // 改变当前点的x和y坐标
        y_pos = y;

        show();                 // 在新位置显示点

}
```

继承机制的应用举例——图形的处理

CIRCLE.CPP

```
#include "basgraph.h"

CIRCLE::CIRCLE(int x, int y, int r): POINT(x, y)
{
        radius = r;
}

void CIRCLE::show()
{
      if (! is_visible())
     {
        visible = TRUE;    // 改变圆的内部状态
        // 画圆，(x_pos, y_pos)为圆心、radius为半径
            circle(x_pos, y_pos, radius);
     }
}
```

继承机制的应用举例——图形的处理

CIRCLE.CPP

```
void CIRCLE::hide()
{
     unsigned int temp_color;  // 用于保存当前颜色的临时变量

     if (is_visible())
    {
          temp_color = getcolor();  // 保存当前的缺省颜色
          setcolor(getbkcolor());    // 设置当前颜色为背景颜色
          visible = FALSE;           // 改变圆的内部状态
          circle(x_pos, y_pos, radius);// 用背景颜色画圆，即擦除圆
          setcolor(temp_color);                  // 恢复原来的缺省颜色
     }
}
```

继承机制的应用举例——图形的处理

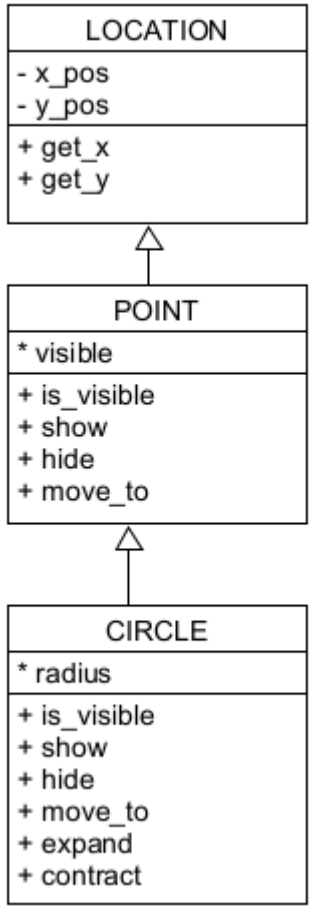
CIRCLE.CPP

```
void CIRCLE::move_to(int x, int y)
{
       hide();   // 擦除旧的圆
       x_pos = x;   // 设置新的位置
       y_pos = y;
       show();         // 在新的位置画圆
}
void CIRCLE::expand(int delta)
{
       hide();  // 擦除旧的圆
       radius = radius + delta;           // 扩大半径
       if (radius < 0)  radius = 0;       // 避免半径为负数
       show();        // 按新的半径画圆
}
void CIRCLE::contract(int delta)
{
      expand(-delta);   // 利用expand()成员函数实现contract()
}
```

继承机制的应用举例——图形的处理

GRAFDEMO.CPP

```
#include "basgraph.h"  // 基本图形元素的类界面
#include <conio.h>     // 利用其中的getch()函数暂停
int main()
{
    int graphdriver = DETECT, graphmode ; // 初始化图形系统所需变量
   // 声明一个圆，圆心在(100, 200)，半径为50
   CIRCLE circle(100, 200, 50);
   initgraph(&graphdriver, &graphmode, ""); // 初始化图形系统
   circle.show();  // 声明一个圆并显示它
   circle.move_to(200, 250);  // 移动圆
   circle.expand(50); // 放大圆
   circle.expand(50);
   circle.contract(65); // 缩小圆
   circle.contract(65);
   closegraph();  // 关闭图形系统
   return 0;
}      //注：此代码需要电脑事先安装好图形相关库环境，请有兴趣同学课下自主运行
```



继承对象的重定义

- 派生类中修改继承成员函数的语义（即，修改函数体，而保持函数原型不变）。
- 派生类中的名字支配（屏蔽）基类中的名字。

```
// 说明：类POINT描述了某一个位置是隐藏的还是显示的。
// 以public继承表示x_pos和y_pos在POINT中是
protected
class POINT: public LOCATION {
public:
        :
       void show();      // 在当前位置显示点
       void hide();        // 将点隐藏起来
       // 将当前点移动到新位置(x, y)并显示它
       void move_to(int x, int y);
protected:
       :
};
```

```
// 说明：类CIRCLE描述了一个在屏幕上由POINT派生出来的圆。
// 由POINT类派生，从而也继承了LOCATION类
class CIRCLE: public POINT {
public:
                                    :
       void show(); // 在屏幕上画出圆
       void hide(); // 将圆隐藏起来
       void move_to(int x, int y); // 将当前圆移动到新位置(x, y)
                                :
protected:
                                    :
};
```

继承对象的重定义

- 派生类中修改继承成员函数的语义（即，修改函数体，而保持函数原型不变）。
- 派生类中的名字支配（屏蔽）基类中的名字。

```
void POINT::show()
{
       if (! is_visible()) {
           visible = TRUE;
         putpixel(x_pos, y_pos, getcolor());
       }
}

void CIRCLE::show()
{
      if (! is_visible())
     {
         visible = TRUE;
           circle(x_pos, y_pos, radius);

     }
}
```

```
#include "basgraph.h"      // 基本图形元素的类界面
#include <conio.h>         // 利用其中的getch()函数暂停

int main()
{
                                                    :
   // 声明一个圆，圆心在(100, 200)，半径为50
   POINT point( 20, 10 );
   CIRCLE circle(100, 200, 50);

   circle.show();  // 显示圆
   point.show();  // 显示点

   circle.move_to(200, 250);  // 移动圆
   point.move_to(100,20);     // 移动点
                                                    :
}
```